Thread solutions

OS04-E01 Sharing

1. Abstract counter++ loads, adds, and stores. Both can read 0 then store 1. In C/C++, unsynchronized non-atomic conflicts are undefined behavior; this trace is not every language-permitted outcome.
2. TLS gives independent counters, removing shared writes but losing one directly shared total. Collect and merge separately.
3. One mutex around the full RMW serializes it and produces 2, provided all accesses obey the same lock. Conflicting accesses outside it can still race.

Judge conflicts on the same memory, synchronization, and lifetime.

OS04-E02 Concurrency and speedup

1. Speedup$\leq 1/(0.2+0.8/8)=1/0.3\approx 3.33$.
2. Synchronization, communication, imbalance, coherence, launch costs, and nonparallel I/O add work or effective serial time under this model.
3. $1/(0.05+0.95/8)=1/0.16875\approx 5.93$.

The upper bound assumes the same problem and unchanged modeled costs.

OS04-E03 Cancellation and cleanup

1. Arbitrary cancellation can interrupt a held lock, unregistered cleanup, or partially updated shared structure. Resources or invariants remain stranded.
2. Deferred cancellation acts only at specified points, which must be cleanable or support rollback. Cond-wait cancellation reacquires its mutex before cleanup. Request success is not completion.
3. Track ownership across normal/error/cancel paths. Disable cancellation while registering handlers. Free B and unlock M exactly once on each path. Join before freeing shared arguments. A finite timeout does not prove exit.

Cancellation is a concurrent transition with an explicit exit/cleanup contract.

Thread solutions (continued)

OS04-E04 M:N compensation

1. One blocked carrier leaves 9, over which 99 other user threads can rotate. Ten blocked carriers leave 0; the remaining 90 have not issued their calls and wait in the runtime ready queue.
2. The limit assumes exactly ten carriers without replacement or async. Replacements K11…K20 can issue more calls. Async can leave 100 tasks awaiting completion while no carrier blocks. User-level waiting differs from kernel blocking.
3. One blocked M:1 carrier prevents scheduling other user threads there. In 1:1, another ready kernel entity can run; parallelism is still bounded by cores and readiness.

Keep the carrier model explicit.

OS04-E05 Creation, join, and TLS

1. Initialize input before create, retain Work until successful join, and read 42 afterward. A sleep is not synchronization; the worker may not have run. Conversely, create success does not imply it has not yet started.
2. Pthread_join requires a joinable target and uses its returned error number. Windows requires a successful signaled wait, not timeout/WAIT_FAILED; CloseHandle still applies and does not kill the thread. Java start creates execution, direct run does not; InterruptedException does not prove completion.
3. Both are unsafe. The stack object has expired, or the detached worker can still access freed arguments. Synchronization cannot revive invalid objects. Detach needs another ownership/completion protocol.
4. Two TLS counters independently go 0→1; main can merge to 2 after join. TLS persists with the worker across function calls and pool jobs. Reset job state explicitly; thread identity is not job identity.

Completion, publication, and lifetime must all hold.

Thread solutions (continued)

OS04-E06 Amdahl, DAG, and false sharing

1. T8=20+80/8=30 ms; speedup=10/3; efficiency=5/12. Infinite-core speedup≤5. Ten extra overhead milliseconds make T8=40 and speedup 2.5.
2. Eighty 1-cycle operations plus twenty 10-cycle operations cost 280. Accelerating the first group eightfold gives 210 and speedup 4/3. Count fractions cannot replace time fractions.
3. Leaves 1…8, pairs 3/7/11/15, quartets 10/26, root 36. Eight leaves plus seven merges give work 15, span 4. Thus $T_P \geq \max(15/P,4)$, speedup≤15/4. Average parallelism is not a hard no-benefit threshold at 3.75 processors.
4. Distinct integers without other conflicts can avoid a language data race while sharing a coherence line. Padding/alignment can reduce that interference, but no improvement factor is claimed without measurement. Padding does not repair conflicting accesses to one counter.

Keep operation counts, timing, and correctness separate.

OS04-E07 Pool and serial-queue waits

1. A/B occupy both workers while their ready children require free workers. Resource waiting deadlocks despite acyclic data dependencies.
2. Use a runtime with explicit help-join, so waiters execute dependencies, or replace same-pool blocking with continuations/external coordination. A larger pool fixes this instance but not arbitrary nesting.
3. The child waits for the parent block to leave Q, while the parent waits for the child. Serial-queue ordering protects only that queue, not shared objects across queues.
4. Private sums 3/7/11/15 combine to 10/26 then 36. Plain unsynchronized sum+= is not reduction. Floating-point reassociation can change final bits; state the numerical contract.

Task dependencies and execution-resource dependencies are different.

Thread solutions (continued)

OS04-E08 Cancellation, fork, and signals

1. No. A request can remain pending while cancellation is disabled or before a deferred point. Successful join establishes the corresponding completion contract; do not free arguments first.
2. Wait atomically releases M; cancellation reacquires M; LIFO cleanup frees B once, then unlocks M once. Normal paths pair pops/cleanup. Disable cancellation during registration and avoid points while abstract invariants are broken.
3. No. Unlocking can expose corrupted shared state. Cancel only at stable boundaries or implement real rollback. Arbitrary asynchronous cancellation is especially unsafe.
4. Child retains only T1. T2's copied lock can remain owned without a surviving T2. Before exec, obey async-signal-safe restrictions and coordinated atfork rules. Child inherits dispositions and T1's signal mask; pending signals start empty. Dispositions are generally process-wide, masks per thread. Process-directed delivery is not once per thread; synchronous faults target the causing thread.

Ownership cleanup does not automatically restore abstract data invariants.

OS04-E09 Historical object counts

1. CreateProcessW includes one initial thread; CreateThread adds one. Total 2 threads, each ETHREAD/KTHREAD/TEB and separate registers/stacks, sharing one process/AS. Two further CreateThreads would instead give 3.
2. Ordinary fork gives 2 tasks, 2 mm, 2 page tables; copied fd tables can still reference the same open descriptions. The specified clone gives 2 tasks/kernel stacks but one mm/page-table context, files table, and sighand. THREAD shares TGID without merging tasks or TIDs.
3. No. Those flags share contexts without CLONE_THREAD's TGID semantics. In v3.2, THREAD requires SIGHAND, and SIGHAND requires VM. Pthreads additionally needs TLS/termination mechanisms; four flags are not a complete implementation.
4. Shared backing enables access, not atomicity. Conflicting writes, publication, and lifetime still need synchronization.

These are the stated Windows 7 and Linux 3.2 models.

Thread solutions (continued)

Verification code appendix: thread_models.py

```
"""Exact arithmetic and sequential teaching models; not hardware benchmarks."""
from fractions import Fraction


def amdahl(baseline,serial_fraction,processors,overhead=0):
    if baseline<=0 or not 0<=serial_fraction<=1 or processors<1 or overhead<0:
        raise ValueError("invalid time model")
    time=baseline*(serial_fraction+(1-serial_fraction)/processors)+overhead
    speedup=baseline/time
    return time,speedup,speedup/processors


def reduce_tree(values):
    """Power-of-two leaves, one unit per leaf and binary merge."""
    if not values or len(values)&(len(values)-1):
        raise ValueError("nonempty power-of-two input required")
    level=list(values);levels=[level];work=len(level);span=1
    while len(level)>1:
        level=[level[i]+level[i+1] for i in range(0,len(level),2)]
        levels.append(level);work+=len(level);span+=1
    return {"levels":levels,"result":level[0],"work":work,"span":span}


def carriers(initial,blocked,replacement=False,maximum=None):
    maximum=initial if maximum is None else maximum
    if initial<1 or maximum<initial or blocked<0:
        raise ValueError("invalid carrier configuration")
    configured=min(initial+blocked,maximum) if replacement else initial
    if blocked>configured:raise ValueError("more blocked carriers than exist")
    return {"configured":configured,"blocked":blocked,"runnable":configured-blocked}


if __name__=="__main__":
    for overhead in (0,10):
        print("Amdahl:",amdahl(100,Fraction(1,5),8,overhead))
    print("Reduction:",reduce_tree(list(range(1,9))))
    print("Fixed:",carriers(10,10))
    print("Compensated:",carriers(10,10,True,20))
```

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study